

Ejercicio 04: Hooks Genéricos

Objetivo

Implementar custom hooks utilizando **generics de TypeScript** para crear hooks reutilizables y type-safe que funcionen con cualquier tipo de dato.

Conceptos Cubiertos

- Generics en custom hooks
- Type inference con generics
- Serialización/deserialización tipada
- Hooks para operaciones asíncronas genéricas
- Patrones de TypeScript avanzados

Estructura

```
ejercicio-04-hooks-genericos/  
├── README.md  
├── starter/  
│   ├── useLocalStorage.ts    # Hook genérico para localStorage  
│   ├── useAsync.ts          # Hook genérico para operaciones async  
│   └── App.tsx               # Demo de uso  
└── solution/  
    ├── useLocalStorage.ts  
    ├── useAsync.ts  
    └── App.tsx
```

Duración Estimada

45 minutos

Hooks a Implementar

useLocalStorage

Hook genérico para persistir estado en localStorage con serialización automática.

Características:

- Genérico para cualquier tipo serializable
- Serialización/deserialización automática con JSON
- Valor inicial con lazy initialization
- Manejo de errores de parsing
- Sincronización entre pestañas (storage event)

Interface:

```
interface UseLocalStorageReturn<T> {  
  value: T;  
  setValue: (value: T | ((prev: T) => T)) => void;  
  removeValue: () => void;  
}
```

useAsync

Hook genérico para manejar operaciones asíncronas con estados de loading, error y data.

Características:

- Acepta cualquier función que retorne Promise
- Estados: idle, loading, success, error
- Función execute para disparar la operación
- Reiniciar estado con reset

Interface:

```
type AsyncStatus = 'idle' | 'loading' | 'success' | 'error';

interface UseAsyncReturn<T> {
  data: T | null;
  status: AsyncStatus;
  error: Error | null;
  isLoading: boolean;
  isError: boolean;
  isSuccess: boolean;
  execute: (...args: unknown[]) => Promise<T | null>;
  reset: () => void;
}
```

Instrucciones

Paso 1: useLocalStorage (20 min)

1. Abre `starter/useLocalStorage.ts`
2. Lee los comentarios explicativos
3. Descomenta el código de la interfaz
4. Descomenta la implementación del hook
5. Observa el uso de generics `<T>`

Paso 2: useAsync (15 min)

1. Abre `starter/useAsync.ts`
2. Implementa los tipos de estado
3. Descomenta el hook genérico
4. Observa cómo el tipo T fluye a través del hook

Paso 3: Demo en App (10 min)

1. Abre `starter/App.tsx`
2. Descomenta los demos
3. Observa la inferencia de tipos automática
4. Prueba diferentes tipos de datos

Criterios de Éxito

Criterio	Peso
useLocalStorage funciona con diferentes tipos	30%
Serialización/deserialización correcta	20%
useAsync maneja todos los estados	25%
TypeScript infiere tipos correctamente	15%

Tips

- Los generics permiten que TypeScript infiera el tipo del valor inicial
- Usa `JSON.stringify` y `JSON.parse` para serialización
- Envuelve operaciones de `localStorage` en `try/catch`
- El tipo `T` en `useState<T>` debe coincidir con el genérico del hook

Recursos

- [TypeScript Generics](#)
- [React + TypeScript Generics](#)